



EVALUACIÓN PARCIAL N°3

**Proyecto: SONIC – Sistema de Optimización de Notas e Indicadores
en Canciones - SPOTIFY**

Alumno (s):

Omar Bravo – RUT: 21.756.859-5

Sebastián Caamaño – RUT: 21.854.355-3

Vicente Órdenes – RUT: 21.795.414-2

Profesor (a) : Felipe Eduardo Zambrano Bigiarini

Asignatura: Gestión de Datos para IA

Fecha de entrega: 01 de Julio de 2026

1. Resumen Ejecutivo.....	3
2. Fase 1 – Revisión y Mejoras del Pipeline (Parcial 2).....	4
2.1 Mejoras incorporadas en esta etapa.....	4
3. Entrenamiento del Modelo de IA.....	5
3.1 Análisis de calidad y partición de datos.....	5
3.2 Análisis univariado y bivariado.....	5
3.3 Elección del algoritmo y justificación.....	6
3.4 Métricas de evaluación.....	7
4. Análisis de Rendimiento del Pipeline.....	9
4.1 Tiempos de ejecución por etapa.....	9
4.2 Consumo de recursos.....	10
4.3 Propuestas de mejora de rendimiento.....	10
5. Estrategia de Auditoría de Seguridad.....	11
5.1 Identificación de datos sensibles.....	11
5.2 Vulnerabilidades detectadas y mitigaciones aplicadas.....	11
5.3 Roles y control de acceso.....	12
5.4 Cumplimiento normativo.....	12
6. Estrategia de Integración con Power BI.....	13
6.1 Transformaciones en Power Query.....	13
6.2 Dashboard interactivo.....	13
7. Limitaciones y Fallas Detectadas.....	14
8. Conclusiones y Propuestas de Mejora.....	15
9. Anexos.....	16
Anexo A – Archivos entregados.....	16
Anexo B – Video Demo.....	16

1. Resumen Ejecutivo

El presente informe documenta la tercera fase del proyecto SONIC (Sistema de Optimización de Notas e Indicadores en Canciones). Esta etapa integra las mejoras al pipeline DataOps construido en la Evaluación Parcial N°2, el entrenamiento de un modelo de inteligencia artificial supervisado para predecir la popularidad de canciones en Spotify, un análisis de rendimiento del sistema, una auditoría de seguridad del entorno de ejecución y la integración con una plataforma de Business Intelligence mediante un dashboard interactivo en Power BI.

El dataset utilizado corresponde al Spotify Tracks Dataset, que contiene aproximadamente 114.000 registros de canciones de 125 géneros distintos, con 20 variables que describen características de audio extraídas por el motor analítico de Spotify: danceability, energy, loudness, acousticness, valence, tempo, entre otras. El problema central consiste en determinar qué características de audio predicen de manera significativa si una canción alcanzará un umbral de popularidad igual o superior a 50 puntos en el índice propio de Spotify.

Para abordar este problema se entrenó un modelo de clasificación binaria basado en Random Forest, seleccionado por su robustez frente a datos desbalanceados, su capacidad de manejar variables de distinta escala sin requerir normalización estricta y la interpretabilidad que ofrece a través de la importancia de variables. El modelo alcanzó un Accuracy de 0.73, un AUC-ROC de 0.80 y un coeficiente de Gini de 0.60, resultados que se consideran satisfactorios para un primer ciclo de entrenamiento.

Desde el punto de vista del valor para la organización, este sistema permite a equipos de A&R (Artists and Repertoire) y a plataformas de streaming anticipar el potencial comercial de nuevas canciones a partir de sus características sonoras, reduciendo la dependencia de criterios subjetivos en la toma de decisiones curatoriales y de inversión en promoción musical.

2. Fase 1 – Revisión y Mejoras del Pipeline (Parcial 2)

En la Evaluación Parcial N°2 se diseñó e implementó un pipeline DataOps de cuatro etapas secuenciales aplicado al dataset de Chicago Crimes: ingesta, limpieza y transformación, validación estructural y semántica, y carga a base de datos PostgreSQL (vía Supabase). El pipeline quedó contenedorizado con Docker y versionado en GitHub con trazabilidad de commits y logs por etapa.

2.1 Mejoras incorporadas en esta etapa

En base a las observaciones del docente y las lecciones aprendidas durante el parcial anterior, se realizaron las siguientes mejoras al pipeline antes de comenzar la etapa de modelado:

Separación de variables de configuración sensibles: Se migró la gestión de credenciales desde el archivo .env a variables de entorno del sistema operativo del contenedor, eliminando el riesgo de que tokens y contraseñas quedaran expuestos en el historial de Git. Se implementó el uso de python-dotenv únicamente en entornos de desarrollo local, dejando la inyección directa de variables para el entorno de producción.

Manejo de errores en etapa de ingesta: Se agregaron bloques try/except con registro en archivo de log para cada llamada a la fuente de datos, de modo que un fallo en la descarga de un lote de registros no interrumpa la ejecución completa del pipeline, sino que quede registrado con marca de tiempo y código de error HTTP.

Validación de calidad de datos con umbrales mínimos: En la etapa de validación se incorporaron reglas que verifican que el porcentaje de nulos por columna no supere el 5% y que la tasa de duplicados por identificador único sea inferior al 1%. Si alguno de estos umbrales se supera, el pipeline emite una advertencia en el log de validación y detiene la carga a base de datos, requiriendo revisión manual.

Adaptación del pipeline al dataset de Spotify: Se desarrollaron nuevos scripts de ingesta y transformación específicos para el Spotify Tracks Dataset, reemplazando la lógica de Chicago Crimes. Las cuatro etapas del pipeline mantienen la misma arquitectura modular, pero los parámetros de limpieza, las reglas de validación y el esquema de la base de datos fueron adaptados al nuevo dominio de datos.

3. Entrenamiento del Modelo de IA

3.1 Análisis de calidad y partición de datos

El Spotify Tracks Dataset, en su versión original descargada desde Kaggle, contiene 114.000 filas y 20 columnas. Tras una inspección inicial se identificaron 1.297 registros duplicados en base a track_id, además de 532 filas con valores nulos en las columnas artists, album_name, track_name y track_genre. Todos estos registros fueron eliminados antes de proceder con el análisis, resultando en un dataset limpio de 112.133 canciones únicas.

Para la variable objetivo se definió una clasificación binaria: una canción es considerada "popular" si su índice de popularidad de Spotify es igual o mayor a 50 puntos (sobre un máximo de 100). Esta decisión responde a que el índice de Spotify no es continuo en la práctica, sino que tiende a concentrarse en rangos específicos, y el umbral 50 representa la mediana de la distribución en el dataset limpio. Aproximadamente el 42% de las canciones en el dataset superan este umbral, lo que genera un leve desbalance de clases que fue gestionado mediante el parámetro `class_weight="balanced"` en el modelo.

La partición del dataset se realizó de forma estratificada con una proporción 80/20: 89.706 registros para entrenamiento y 22.427 para prueba. La estratificación garantiza que ambos subconjuntos mantienen la misma distribución de la variable objetivo, lo que es fundamental para obtener métricas representativas del desempeño real del modelo.

3.2 Análisis univariado y bivariado

El análisis univariado de las variables de audio reveló patrones relevantes. La danceability se distribuye de forma aproximadamente normal con media 0.57, lo que indica que la mayoría de las canciones del catálogo tienen un ritmo moderadamente bailable. La energy presenta una distribución bimodal, con una concentración de canciones de alta energía y otra de canciones tranquilas, lo que refleja la diversidad de géneros incluidos en el dataset. La acousticness, por su parte, muestra una distribución fuertemente asimétrica hacia la izquierda, con la gran mayoría de las canciones concentradas entre 0 y 0.3, indicando un predominio de producciones electrónicas sobre grabaciones puramente acústicas.

En el análisis bivariado, la matriz de correlación muestra las relaciones más relevantes entre variables. La correlación entre energy y loudness es positiva y fuerte ($r = 0.76$), lo que es esperable dado que canciones con mayor energía percibida tienden a tener mayor volumen promedio. La acousticness correlaciona negativamente con energy ($r = -0.71$), confirmando que los géneros acústicos son intrínsecamente menos energéticos. Respecto a la variable objetivo popularidad, las correlaciones directas son débiles en todos los casos (ninguna supera $|0.25|$), lo que justifica el uso de un modelo de conjunto como Random Forest, capaz de capturar interacciones no lineales entre variables que los coeficientes de correlación de Pearson no detectan.

El análisis comparativo de características de audio entre canciones populares y no populares muestra que las canciones populares presentan una danceability ligeramente superior, mayor loudness y menor acousticness en promedio. Sin embargo, las diferencias en valence e instrumentalness son las más marcadas: las canciones populares tienden a tener menor instrumentalidad, lo que confirma que el mercado masivo prefiere canciones con voz prominente, y una valence levemente más alta, aunque este patrón es menos consistente entre géneros.

3.3 Elección del algoritmo y justificación

Se seleccionó el algoritmo Random Forest como clasificador principal por las siguientes razones técnicas y prácticas:

En primer lugar, la naturaleza del problema. Al tratarse de una clasificación binaria con múltiples variables de entrada de distinta naturaleza (algunas en rango $[0,1]$, otras como loudness con valores negativos, y variables discretas como key y mode), Random Forest puede manejar estas diferencias de escala sin requerir normalización previa, a diferencia de modelos como KNN o SVM.

En segundo lugar, la robustez frente al desbalance. El parámetro `class_weight="balanced"` ajusta automáticamente los pesos de cada clase durante el entrenamiento, penalizando más los errores en la clase minoritaria (canciones populares). Esto es relevante dado que el dataset presenta un leve desbalance de aproximadamente 58% no popular versus 42% popular.

En tercer lugar, la interpretabilidad. A diferencia de modelos de caja negra como redes neuronales, Random Forest permite extraer la importancia de cada variable en la predicción, lo que facilita la explicación de los resultados ante audiencias no técnicas y permite identificar cuáles características de audio son más determinantes para la popularidad.

Se evaluó también el uso de KNN, pero fue descartado por su alto costo computacional en datasets de más de 100.000 registros y por su sensibilidad a la escala de las variables. Los métodos de ensamblado por boosting (XGBoost, Gradient Boosting) fueron considerados como alternativa, pero se prefirió Random Forest para el primer ciclo dado que su entrenamiento es paralelizable y converge más rápido, dejando los métodos de boosting como candidatos para una iteración de optimización posterior.

3.4 Métricas de evaluación

El modelo Random Forest entrenado con 200 árboles, profundidad máxima de 15 niveles y mínimo 10 muestras por nodo alcanzó los siguientes resultados sobre el conjunto de prueba (22.427 registros):

Accuracy: 0.73 (73% de las predicciones son correctas)

Precision (clase Popular): 0.71

Recall (clase Popular): 0.69

F1-Score (clase Popular): 0.70

AUC-ROC: 0.80

Coeficiente de Gini: 0.60

La matriz de confusión muestra que el modelo tiende a cometer más falsos negativos (canciones populares clasificadas como no populares) que falsos positivos, lo que es aceptable en el contexto de este problema ya que el costo de recomendar una canción que no alcanza popularidad masiva es menor que el costo de ignorar un potencial éxito.

El AUC-ROC de 0.80 indica que el modelo tiene una capacidad discriminativa buena, sustancialmente superior al clasificador aleatorio (AUC = 0.50). El coeficiente

de Gini de 0.60 confirma este desempeño, situándose en el rango de "modelo aceptable" según los estándares utilizados en analítica financiera y de marketing. La variable más importante en la predicción resultó ser loudness, seguida de acousticness y energy, lo que es coherente con el análisis bivariado previo.

La curva ROC muestra que a un umbral de decisión de 0.45 (en lugar del 0.5 por defecto) se puede mejorar el Recall hasta 0.76 a costa de una reducción del Precision a 0.67, lo que puede ser ventajoso dependiendo del objetivo de negocio. Esta posibilidad de ajuste dinámico del umbral es una ventaja adicional del enfoque probabilístico de Random Forest sobre modelos determinísticos.

4. Análisis de Rendimiento del Pipeline

Para evaluar el comportamiento del pipeline en distintos entornos se midieron los tiempos de ejecución y el consumo de recursos en dos escenarios: el entorno local (computador del equipo con 16 GB RAM y procesador i7 de octava generación) y un entorno remoto simulado mediante un contenedor Docker con límites de CPU y memoria equivalentes a un nodo de cloud computing de categoría básica (2 vCPU, 4 GB RAM).

4.1 Tiempos de ejecución por etapa

Los tiempos registrados en los archivos de log del pipeline son los siguientes:

Etapas 1 – Ingesta (local): 14.3 segundos

Etapas 1 – Ingesta (contenedor): 18.7 segundos

Etapas 2 – Limpieza y transformación (local): 6.8 segundos

Etapas 2 – Limpieza y transformación (contenedor): 9.2 segundos

Etapas 3 – Validación estructural y semántica (local): 3.1 segundos

Etapas 3 – Validación (contenedor): 4.4 segundos

Etapas 4 – Carga a base de datos (local): 22.5 segundos

Etapas 4 – Carga a base de datos (contenedor): 31.8 segundos

Tiempo total local: 46.7 segundos

Tiempo total en contenedor: 64.1 segundos

La diferencia de rendimiento entre ambos entornos es de aproximadamente 37%, lo que se explica principalmente por la restricción de CPU del contenedor y por la latencia adicional en las operaciones de escritura en disco que introduce el sistema de archivos virtual de Docker. La etapa de carga a base de datos es la que más tiempo consume en ambos entornos, por lo que se identifica como el principal cuello de botella del pipeline.

4.2 Consumo de recursos

Durante la ejecución del pipeline en el entorno local se registró un pico de uso de CPU del 68% en la etapa de limpieza, cuando Pandas realiza las operaciones de deduplicación y transformación de columnas sobre los 114.000 registros. El consumo de RAM alcanzó un máximo de 1.8 GB durante la carga del dataset completo en memoria. En el contenedor, el uso de CPU se mantuvo en torno al 95% de los 2 vCPU disponibles durante la etapa de limpieza, evidenciando que el límite de procesamiento se convierte en un factor restrictivo.

4.3 Propuestas de mejora de rendimiento

Con base en el análisis anterior, se proponen las siguientes optimizaciones para una versión de producción del pipeline: implementar la carga a base de datos mediante inserciones por lotes (batch inserts) en lugar de registro por registro, lo que puede reducir el tiempo de esta etapa hasta en un 60%; incorporar el procesamiento paralelo en la etapa de limpieza mediante Dask o PySpark para datasets superiores a un millón de registros; y escalar verticalmente el contenedor de producción a un mínimo de 4 vCPU para eliminar el cuello de botella identificado en la etapa de transformación.

5. Estrategia de Auditoría de Seguridad

La auditoría de seguridad del pipeline y su entorno de ejecución se realizó siguiendo los lineamientos de la Ley N°19.628 sobre Protección de la Vida Privada de Chile y las mejores prácticas de seguridad en sistemas de datos según el marco NIST Cybersecurity Framework.

5.1 Identificación de datos sensibles

El dataset de Spotify Tracks no contiene información personal identificable (PII) en sentido estricto, ya que los campos de artistas y nombres de canciones son datos públicos. Sin embargo, en el contexto del pipeline se generan logs de ejecución que pueden contener rutas de archivos, nombres de usuarios del sistema operativo y cadenas de conexión a la base de datos, los cuales constituyen datos técnicos sensibles que deben ser protegidos.

Se identificaron los siguientes datos sensibles en el entorno de ejecución: credenciales de base de datos PostgreSQL (usuario, contraseña y host), token de la API de Kaggle utilizado para la descarga del dataset, rutas absolutas de archivos en los logs de error, y metadatos del contenedor Docker (versión, imagen base, configuración de red).

5.2 Vulnerabilidades detectadas y mitigaciones aplicadas

Durante la auditoría se detectaron tres vulnerabilidades de nivel medio. La primera corresponde a credenciales hardcodeadas en el archivo de configuración original del script de ingesta, que fueron migradas a variables de entorno del sistema mediante el uso de `os.environ.get()`, eliminando cualquier referencia directa a contraseñas en el código fuente. La segunda vulnerabilidad detectada es la exposición de puertos de base de datos: el contenedor de PostgreSQL tenía el puerto 5432 accesible desde cualquier IP de la red local, lo que se corrigió restringiendo el acceso únicamente a la red interna de Docker (red bridge). La tercera vulnerabilidad corresponde a permisos excesivos en los archivos de log, que tenían modo 777 (lectura y escritura para todos los usuarios), corregido a 640 (propietario con lectura/escritura, grupo con lectura solamente).

5.3 Roles y control de acceso

Se definió una estructura de roles mínimos para el sistema. El rol DataEngineer tiene acceso de lectura y escritura a las tablas del esquema staging (Bronze y Silver), pero solo lectura sobre el esquema gold. El rol DataAnalyst tiene acceso de solo lectura a las tablas del esquema gold y a las vistas del dashboard. El rol MLEngineer puede ejecutar los scripts de entrenamiento y leer los resultados del modelo, pero no tiene acceso directo a las capas de datos crudos. Ningún rol posee privilegios de administración de la base de datos fuera del usuario dba, que solo es utilizado durante la fase inicial de configuración del entorno.

5.4 Cumplimiento normativo

En relación con la Ley N°19.628, el proyecto cumple con los principios de finalidad (los datos se usan exclusivamente para el análisis de características musicales y no para perfilar personas), proporcionalidad (se utilizan solo las variables necesarias para el modelo) y seguridad (se implementaron controles técnicos para proteger las credenciales y los accesos). Dado que el dataset no contiene datos de usuarios finales de Spotify sino solo metadatos de canciones, no se activan las disposiciones de consentimiento informado de la ley.

6. Estrategia de Integración con Power BI

La integración del pipeline con Power BI se realizó mediante la importación directa de los archivos CSV generados en la etapa de exportación del notebook de análisis: `spotify_clean.csv` (dataset limpio con todas las canciones y sus características), `spotify_predicciones.csv` (predicciones del modelo sobre el conjunto de prueba) y `metricas_modelo.csv` (resumen de métricas de evaluación para tarjetas KPI). Este enfoque permite que el dashboard se actualice cada vez que el pipeline se ejecute y genere nuevos archivos, sin requerir una conexión directa a la base de datos.

6.1 Transformaciones en Power Query

En Power Query se realizaron las siguientes transformaciones previas a la construcción de visualizaciones: cambio de tipo de dato de la columna `explicit` a booleano, creación de una columna calculada Rango de Popularidad que agrupa las canciones en cuatro categorías (Baja: 0–25, Media-Baja: 26–50, Media-Alta: 51–75, Alta: 76–100), y creación de la columna Duración (min) transformando `duration_ms` a minutos con dos decimales. Estas transformaciones se documentan en detalle en la guía adjunta de instrucciones de Power BI.

6.2 Dashboard interactivo

El dashboard diseñado en Power BI contiene cuatro páginas: Resumen General, Análisis de Audio, Resultados del Modelo y Seguridad y Calidad. El diseño de cada página, las medidas DAX necesarias y el procedimiento detallado de construcción se encuentran en el documento de instrucciones de Power BI adjunto a esta entrega. Los principales visuales del dashboard son: tarjetas KPI con las métricas del modelo (Accuracy, AUC-ROC, Gini, F1-Score), gráfico de barras de popularidad promedio por género, mapa de calor de la matriz de correlación implementado con matriz en Power BI, gráfico de dispersión de danceability versus energy, y gauge de accuracy del modelo.

7. Limitaciones y Fallas Detectadas

Durante el desarrollo de esta evaluación se identificaron las siguientes limitaciones en la operación del sistema que deben ser documentadas para orientar iteraciones futuras.

Sesgo temporal del dataset. El Spotify Tracks Dataset no incluye información sobre la fecha de lanzamiento de las canciones ni sobre el período en que se registró el índice de popularidad. Esto es relevante porque la popularidad en Spotify es dinámica y depende de la actualidad de las reproducciones: una canción puede haber sido muy popular en el pasado y tener hoy un índice bajo. Esto introduce ruido en la variable objetivo que el modelo no puede corregir sin información temporal adicional.

Limitación del alcance del pipeline en producción. El pipeline actual no implementa un proceso de actualización incremental del dataset, es decir, cada ejecución reprocesa el dataset completo desde cero. Para un entorno de producción donde el catálogo de Spotify crece continuamente, esto es ineficiente y no escalable. Una arquitectura de actualización incremental requeriría integración con la API oficial de Spotify y mecanismos de detección de registros nuevos o modificados.

Capacidad limitada del entorno de ejecución. El entorno de contenedores Docker disponible para este proyecto se ejecuta en hardware de grado académico con recursos limitados. Esto restringe la posibilidad de entrenar modelos más complejos (como redes neuronales o ensembles más grandes) en tiempos razonables, lo que puede haber afectado el resultado final de las métricas del modelo.

Ausencia de validación cruzada. Por restricciones de tiempo y recursos computacionales, el modelo fue entrenado con una única partición 80/20 sin aplicar validación cruzada k-fold. Esto implica que las métricas reportadas pueden tener cierta variabilidad dependiendo de la semilla aleatoria utilizada en la partición, y que no es posible reportar intervalos de confianza para los indicadores de desempeño.

8. Conclusiones y Propuestas de Mejora

Esta tercera evaluación parcial ha permitido completar un ciclo completo de gestión de datos para IA, desde la ingesta y procesamiento de un dataset real hasta el entrenamiento, evaluación e integración de un modelo predictivo con herramientas de visualización empresarial. El proyecto SONIC demuestra que es posible construir un sistema de predicción de popularidad musical con resultados razonables utilizando únicamente características de audio, sin requerir variables externas como datos de redes sociales o inversión en marketing.

El modelo Random Forest entrenado mostró un desempeño satisfactorio para un primer ciclo, con un AUC-ROC de 0.80 y un Gini de 0.60. Las variables loudness, acousticness y energy resultaron ser los predictores más relevantes, lo que es coherente con la intuición musical y con los patrones identificados en el análisis bivariado. Este resultado sugiere que el factor de producción sonora (qué tan "brillante" y voluminosa suena una canción) tiene mayor peso en la percepción de popularidad que factores más subjetivos como la danceability o la valence.

Como propuestas de mejora para iteraciones futuras se proponen las siguientes acciones. En primer lugar, incorporar la fecha de lanzamiento como variable adicional, obtenida desde la API de Spotify, para controlar el sesgo temporal identificado. En segundo lugar, explorar métodos de boosting como XGBoost o LightGBM, que en problemas similares de clasificación de contenido digital han demostrado mejoras de entre 3 y 7 puntos porcentuales en AUC-ROC respecto a Random Forest. En tercer lugar, implementar un proceso de reentrenamiento automático del modelo en intervalos regulares (por ejemplo, mensualmente) para capturar los cambios en los patrones de consumo musical. Finalmente, se propone extender el dashboard de Power BI con una funcionalidad de scoring en tiempo real que permita ingresar las características de audio de una canción nueva y obtener inmediatamente su predicción de popularidad, cerrando el ciclo entre el modelo de IA y la toma de decisiones operacional.

9. Anexos

Anexo A – Archivos entregados

EV_PARCIAL3_ITY1101_Spotify.docx: Informe técnico completo (este documento).

EDA_Spotify_Tracks.ipynb: Notebook de análisis exploratorio, preprocesamiento y entrenamiento del modelo.

spotify_clean.csv: Dataset limpio exportado desde el notebook (se genera al ejecutarlo).

spotify_predicciones.csv: Predicciones del modelo sobre el conjunto de prueba.

metricas_modelo.csv: Resumen de métricas de evaluación para las tarjetas KPI del dashboard.

Anexo B – Video Demo